Hotel Management System
Group 6

by

Shaheer Ahmed (BCS233106)

Usman Ali (BCS233194)

M. Abdul Manan (BCS233209)

Abdul Moeed (BCS233202)

Submitted to: Mam Snober Naseer

Subject: CS2143 Data Structures

Date: 12/1/2025

Capital University of Science and Technology

Department of Computer Science

1. Project Description

This Hotel Management System is a simple C++ program to manage basic hotel operations. It handles room bookings, customer checkouts, keeps a history of reservations, and manages a waiting queue for unavailable rooms. The system uses basic data structures like Binary Search Trees (BST), Linked Lists, and Queues to organize and process data efficiently.

2. Key Features

- **Room Management**:
  - Shows room details, like room number and availability.
  - Lets users book available rooms.
  - Handles room checkouts, making them available for new bookings.
- **Reservation History**:
  - Keeps a record of all past reservations using a linked list.
- **Waiting Queue Management**:
  - Adds customers to a waiting queue if rooms are unavailable.
  - Automatically assigns rooms to customers in the queue when rooms become available.

3. Data Structures Used

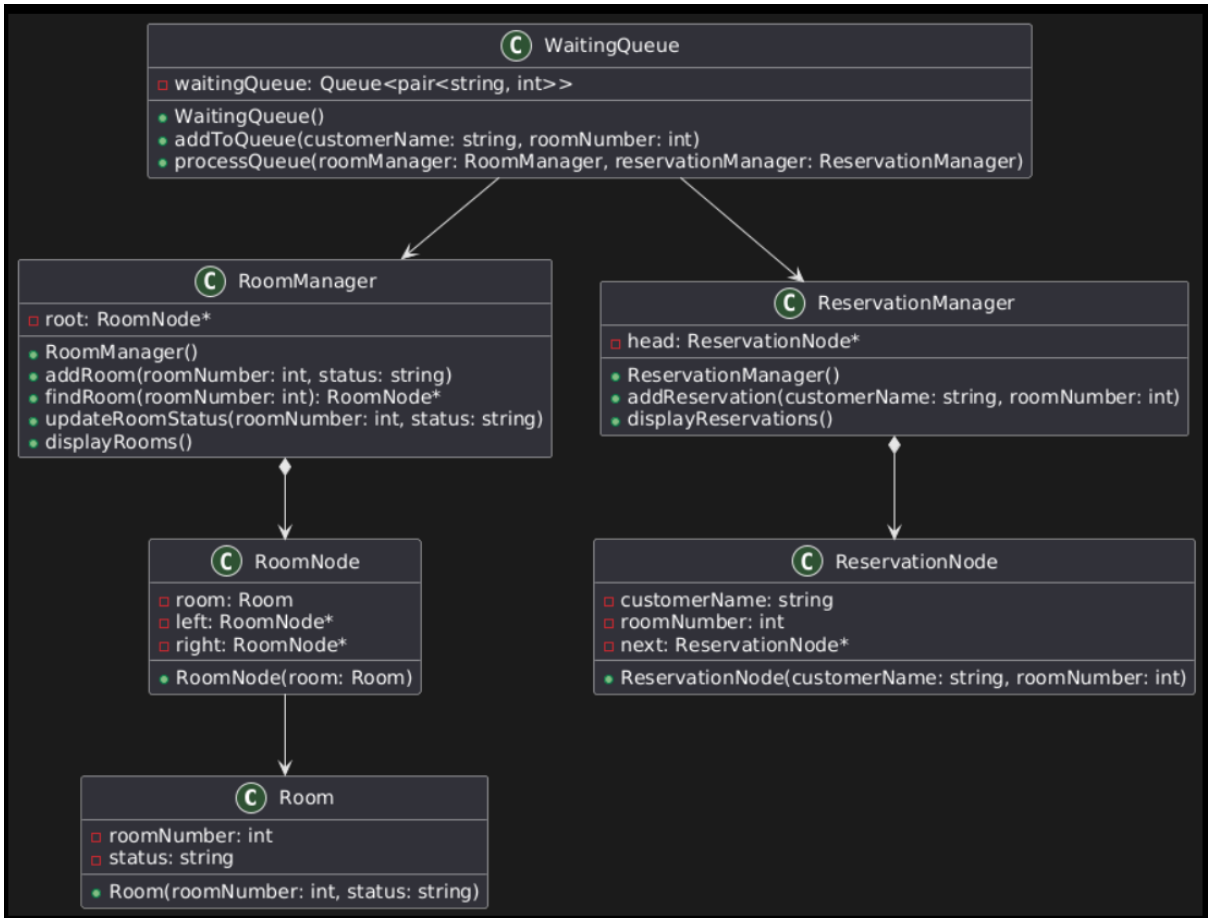
- **Binary Search Tree (BST)**:
  - Each room is a node in the BST. Rooms are stored in an ordered way based on their numbers.
  - This makes searching for a room quick and efficient.

- **Linked List**:
  - Each reservation (customer and room details) is stored as a node in the linked list.
  - New reservations are easily added to the list.
- **Queue**:
  - The queue holds customers waiting for rooms. Each entry includes the customer's name and requested room number.
  - Rooms are assigned to customers in the same order they joined the queue.

4. Program Explanation

- **Room Management**:
  - Rooms are added to the system and stored in the BST.
  - Booking a room updates its status to "Occupied."
  - Checking out updates the room's status back to "Available."
- **Reservation History**:
  - All reservations are stored in a linked list.
  - The list is displayed to show the history of all bookings.
- **Waiting Queue**:
  - Customers who can't get a room are added to the queue.
  - The queue is checked periodically, and rooms are assigned to waiting customers when available.

5. UML Class Diagram



6. Code

```
#include <iostream>
#include <string>
#include <queue>
using namespace std;

// Class to represent a Room
class Room {
public:
    int roomNumber;
    string status; // "Available" or "Occupied"

    Room()
    {
        roomNumber = 0;
        status = "";
```

```
    }
    Room(int roomNo, string Status)
    {
        roomNumber = roomNo;
        status = Status;
    }
};

// Node for Binary Search Tree (BST) to store room details
class RoomNode {
public:
    Room room;
    RoomNode* left;
    RoomNode* right;

    RoomNode(Room r)
    {
        room = r;
        left = NULL;
        right = NULL;
    }
};

// Class to manage the Binary Search Tree (BST) for rooms
class RoomManager {
private:
    RoomNode* root;

    RoomNode* insert(RoomNode* curr, Room room) {
    RoomNode* parent = NULL;
    RoomNode* newNode = new RoomNode(room);
    if (curr == NULL) {
        return newNode;
    }
    RoomNode* temp = curr;
    while (temp != NULL) {
        parent = temp;
        if (room.roomNumber < temp->room.roomNumber) {
            temp = temp->left;
        }
        else {
            temp = temp->right;
        }
    }
    if (room.roomNumber < parent->room.roomNumber) {
        parent->left = newNode;
    }
    else {
        parent->right = newNode;
    }
    return curr;
}
    RoomNode* search(RoomNode* node, int roomNumber) {
        if (node == NULL || node->room.roomNumber == roomNumber) {
            return node;
```

```
        }
        if (roomNumber < node->room.roomNumber) {
            return search(node->left, roomNumber);
        }
        return search(node->right, roomNumber);
    }

    void display(RoomNode* node) {
        if (node == NULL) {
            return;
        }
        display(node->left);
        cout << "Room " << node->room.roomNumber << ": " << node-
>room.status << endl;
        display(node->right);
    }

public:
    RoomManager()
    {
        root = NULL;
    }

    void addRoom(int roomNumber, string status) {
        root = insert(root, Room(roomNumber, status));
    }

    RoomNode* findRoom(int roomNumber) {
        return search(root, roomNumber);
    }

    void updateRoomStatus(int roomNumber, string status) {
        RoomNode* roomNode = findRoom(roomNumber);
        if (roomNode != NULL) {
            roomNode->room.status = status;
        }
        else {
            cout << "Room not found!\n";
        }
    }

    void displayRooms() {
        display(root);
    }
};

// Node for Linked List to store reservation history
class ReservationNode {
public:
    string customerName;
    int roomNumber;
    ReservationNode* next;

    ReservationNode(string custName, int roomNo)
    {
```

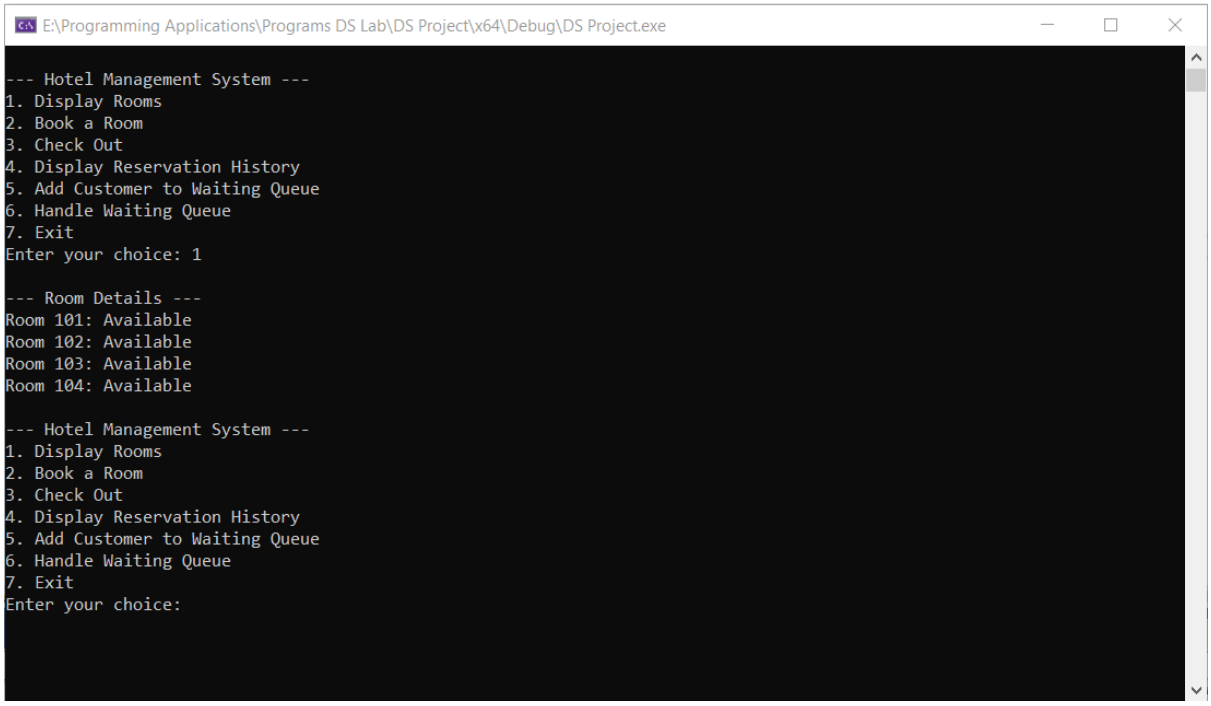
```
        customerName = custName;
        roomNumber = roomNo;
        next = NULL;
    }
};

// Class to manage reservations
class ReservationManager {
private:
    ReservationNode* head;

public:
    ReservationManager()
    {
        head = NULL;
    }

    void addReservation(string customerName, int roomNumber) {
        ReservationNode* newNode = new ReservationNode(customerName,
roomNumber);
        newNode->next = head;
        head = newNode;
        cout << "Reservation added for " << customerName << " in room " <<
roomNumber << ".\n";
    }

    void displayReservations() {
        cout << "\n--- Reservation History ---\n";
        ReservationNode* current = head;
        while (current != NULL) {
            cout << "Customer: " << current->customerName << ", Room: " <<
current->roomNumber << endl;
            current = current->next;
        }
    }
};

// Class to manage the waiting queue
class WaitingQueue {
private:
    queue<pair<string, int>> waitingQueue;

public:
    void addToQueue(string customerName, int roomNumber) {
        waitingQueue.push({ customerName, roomNumber });
        cout << "Customer " << customerName << " added to the waiting
queue for room " << roomNumber << ".\n";
    }

    void processQueue(RoomManager& roomManager, ReservationManager&
reservationManager) {
        while (!waitingQueue.empty()) {
            string customerName = waitingQueue.front().first;
            int requestedRoom = waitingQueue.front().second;
```

```
            RoomNode* roomNode = roomManager.findRoom(requestedRoom);

            if (roomNode != NULL && roomNode->room.status == "Available")
{
                roomManager.updateRoomStatus(requestedRoom, "Occupied");
                reservationManager.addReservation(customerName,
requestedRoom);
                cout << "Customer " << customerName << " has been assigned
room " << requestedRoom << " from the waiting queue.\n";
                waitingQueue.pop();
            }
            else {
                cout << "Room " << requestedRoom << " is still not
available for customer " << customerName << ".\n";
                break;
            }
        }
    }

};

// Main function
int main() {
    RoomManager roomManager;
    ReservationManager reservationManager;
    WaitingQueue waitingQueue;

    // Preload some rooms into the system
    roomManager.addRoom(101, "Available");
    roomManager.addRoom(102, "Available");
    roomManager.addRoom(103, "Available");
    roomManager.addRoom(104, "Available");

    int choice;
    do {
        cout << "\n--- Hotel Management System ---\n";
        cout << "1. Display Rooms\n";
        cout << "2. Book a Room\n";
        cout << "3. Check Out\n";
        cout << "4. Display Reservation History\n";
        cout << "5. Add Customer to Waiting Queue\n";
        cout << "6. Handle Waiting Queue\n";
        cout << "7. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1: {
            cout << "\n--- Room Details ---\n";
            roomManager.displayRooms();
            break;
        }

        case 2: {
            int roomNumber;
```

```
            string customerName;
            cout << "Enter room number to book: ";
            cin >> roomNumber;
            cin.ignore();
            RoomNode* roomNode = roomManager.findRoom(roomNumber);

            if (roomNode != NULL && roomNode->room.status == "Available")
{
                cout << "Enter customer name: ";
                getline(cin, customerName);
                roomManager.updateRoomStatus(roomNumber, "Occupied");
                reservationManager.addReservation(customerName,
roomNumber);
            }
            else {
                cout << "Room is either not available or does not
exist!\n";
            }
            break;
        }


        case 3: {
            int roomNumber;
            cout << "Enter room number to check out: ";
            cin >> roomNumber;
            RoomNode* roomNode = roomManager.findRoom(roomNumber);

            if (roomNode != NULL && roomNode->room.status == "Occupied") {
                roomManager.updateRoomStatus(roomNumber, "Available");
                cout << "Room " << roomNumber << " is now available.\n";
            }
            else {
                cout << "Room is either not occupied or does not
exist!\n";
            }
            break;
        }

        case 4: {
            reservationManager.displayReservations();
            break;
        }

        case 5: {
            string customerName;
            int roomNumber;
            cout << "Enter customer name: ";
            cin.ignore();
            getline(cin,customerName);
            cout << "Enter requested room number: ";
            cin >> roomNumber;
            waitingQueue.addToQueue(customerName, roomNumber);
            break;
        }
```

```
            case 6: {
                waitingQueue.processQueue(roomManager, reservationManager);
                break;
            }

            case 7: {
                cout << "Exiting the system. Goodbye!\n";
                break;
            }

            default: {
                cout << "Invalid choice. Please try again.\n";
            }
            }

        } while (choice != 7);

        return 0;
    }
```

7. Output Screenshots



```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 2
Enter room number to book: 101
Enter customer name: Shaheer Ahmed
Reservation added for Shaheer Ahmed in room 101.

--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 2
Enter room number to book: 102
Enter customer name: Ali Hassan
Reservation added for Ali Hassan  in room 102.

--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 2
Enter room number to book: 103
Enter customer name: Usman Ali
Reservation added for Usman Ali in room 103.

--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 2
Enter room number to book: 104
Enter customer name: Abdul Moeed
Reservation added for Abdul Moeed in room 104.
```

```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 1

--- Room Details ---
Room 101: Occupied
Room 102: Occupied
Room 103: Occupied
Room 104: Occupied
```

```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 4

--- Reservation History ---
Customer: Abdul Moeed, Room: 104
Customer: Usman Ali, Room: 103
Customer: Ali Hassan , Room: 102
Customer: Shaheer Ahmed, Room: 101
```

Microsoft Visual Studio Debug Console

```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 5
Enter customer name: M.Abdul Manan
Enter requested room number: 101
Customer M.Abdul Manan added to the waiting queue for room 101.

--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 6
Room 101 is still not available for customer M.Abdul Manan.
```

Microsoft Visual Studio Debug Console

```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 3
Enter room number to check out: 101
Room 101 is now available.
```

Microsoft Visual Studio Debug Console

```
--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 6
Reservation added for M.Abdul Manan in room 101.
Customer M.Abdul Manan has been assigned room 101 from the waiting queue.

--- Hotel Management System ---
1. Display Rooms
2. Book a Room
3. Check Out
4. Display Reservation History
5. Add Customer to Waiting Queue
6. Handle Waiting Queue
7. Exit
Enter your choice: 7
Exiting the system. Goodbye!
```

8. How the Program Works

- **Initialization**:
  - Rooms are added to the system using the addRoom() method of the RoomManager class.
- **User Interaction**:
  - The user navigates a menu to book rooms, check out, or view records.
- **Room Booking**:
  - Booking updates the room status in the BST and adds the reservation to the linked list.
- **Waiting Queue**:
  - If rooms are unavailable, customer details are added to the queue.
  - Rooms are assigned to customers in the queue when they become available.

9. Conclusion

This Hotel Management System is simple yet effective, demonstrating how data structures like BSTs, linked lists, and queues can be used to solve real-world problems. The system is efficient, flexible, and ensures that operations like room management and customer service are handled smoothly.